

ICT 1306

OBJECT ORIENTED PROGRAMMING

1. INTRODUCTION

TC Irugalbandara
KHA Hettige

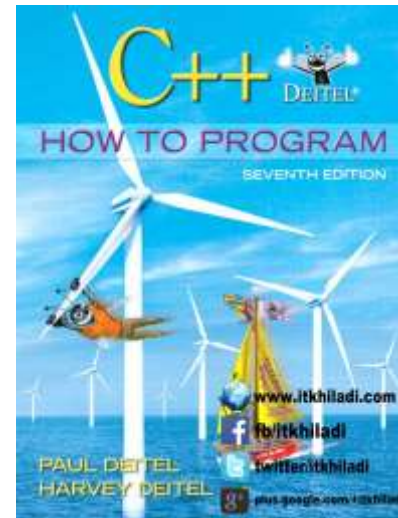
Evaluation Criteria

- Continuous Assessments - 40%
 - Mini Project/Assignments - 30%
 - Mid Examination - 10%
- Final Examination - 60%
 - Theory - 40%
 - Practical - 20%

❖ To be eligible for the exam , all the assignments should be submitted.

References

- Object Oriented Programming with C++, Robert Lafore, 4th Edition.
- C++ How to Program, Deitel and Deitel, 7th Edition.

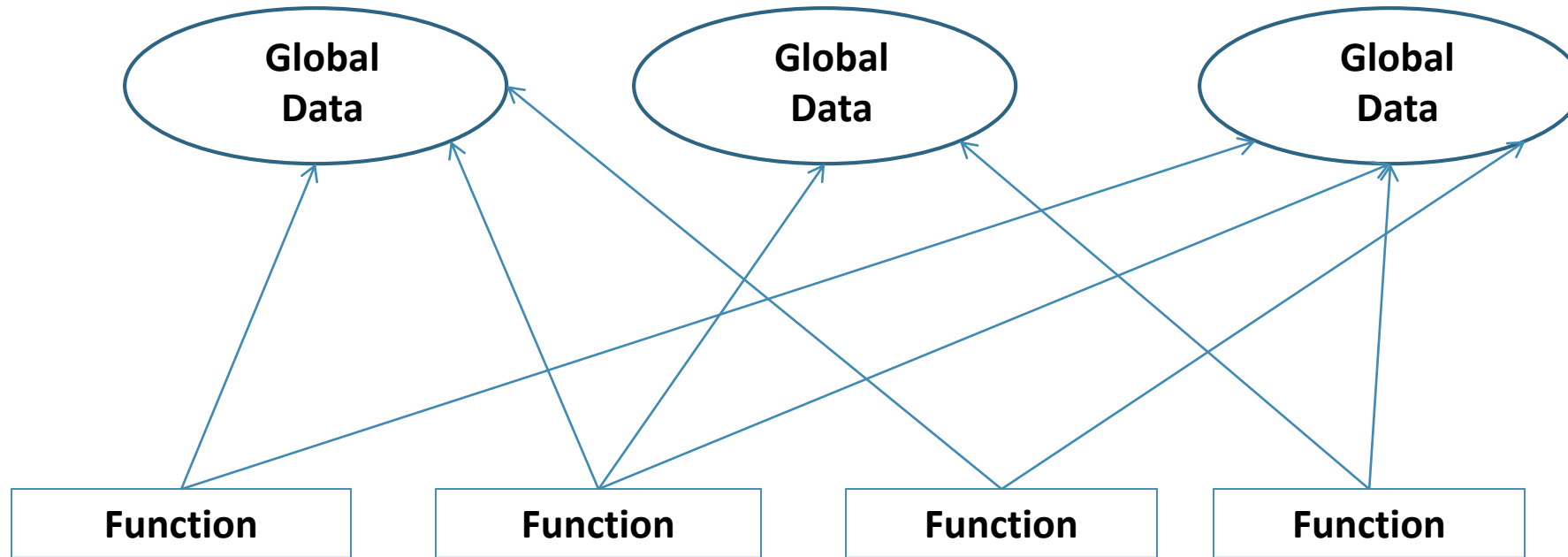


PROCEDURAL VS. OBJECT-ORIENTED LANGUAGES

- Procedural Language
 - Each statement in the program tell the computer to do something
 - Views a program as a series of steps to be carried out/ list of instructions
 - E.g. C, FORTRAN, Pascal
- Structured Programming
 - Complexity → Functions → Module
- Object Oriented Language
 - Views a program as a group of objects that have certain properties and can perform certain functions
 - E.g. C++, Java, C#

- Problems with Procedural Languages
 - Cannot cope with very large project sizes
 - Expensive software errors (e.g. air traffic control)
- Causes of Problems
 - **Unrestricted Access to Data**
 - Global data is allowed
 - Access to data by multiple functions means many connections between functions
 - Programs become difficult to understand, modify and maintain
Ex: Changing a global data item : Student Details
 - **Poor Modeling of Real World Things**
 - Real world things are integral collections of data and functions
 - e.g. a car: has data (make, model etc.) and functions (acceleration)
 - Procedural languages do not tie up data with functions

THE PROCEDURAL PARADIGM



GOAL OF OOP

- Clearer, more reliable, more easily maintained programs
- More effective way of coping with program complexity
- Provide restricted access to data
- Give a real world modeling

Example :Procedural program

- 1. Gather ingredients
 - Flour, butter, egg, sugar etc
- 2. preheat oven to 180 degree
- 3. beat sugar , butter and eggs
- 4. add flour
- 5. mix
- 6. bake 45 mins

Example :Object Oriented programming

- Model for an Object
 - Properties
 - List of ingredients, or set of data
 - Methods
 - List of actions or instructions

Example : Object Oriented programming

- Baker

{

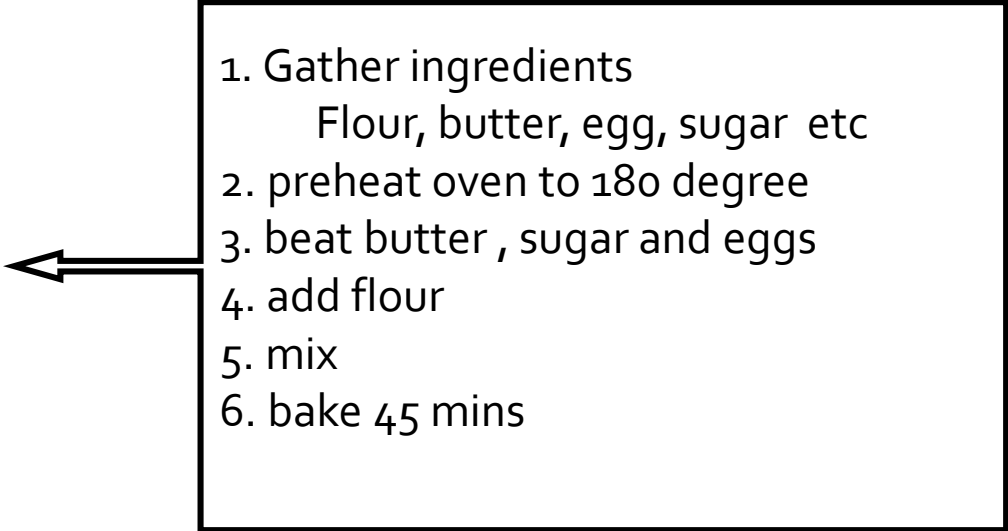
- Properties (ingredients)

- Flour
- Butter
- Eggs
- Cake pan
- Oven

- Methods (actions)

- Bake cookies
- Bake cake
- Bake pie

}

- 
1. Gather ingredients
Flour, butter, egg, sugar etc
 2. preheat oven to 180 degree
 3. beat butter , sugar and eggs
 4. add flour
 5. mix
 6. bake 45 mins

THE OO APPROACH

- Real world Objects have
 - Attributes : characteristics
 - Behaviors : actions
- The fundamental idea is to combine into a single unit both data and functions that operate on the data.
- Such a unit is called an “**Object**”.
- An object’s functions are called “member functions”
- And its data is called “ data members”.

THE OO APPROACH

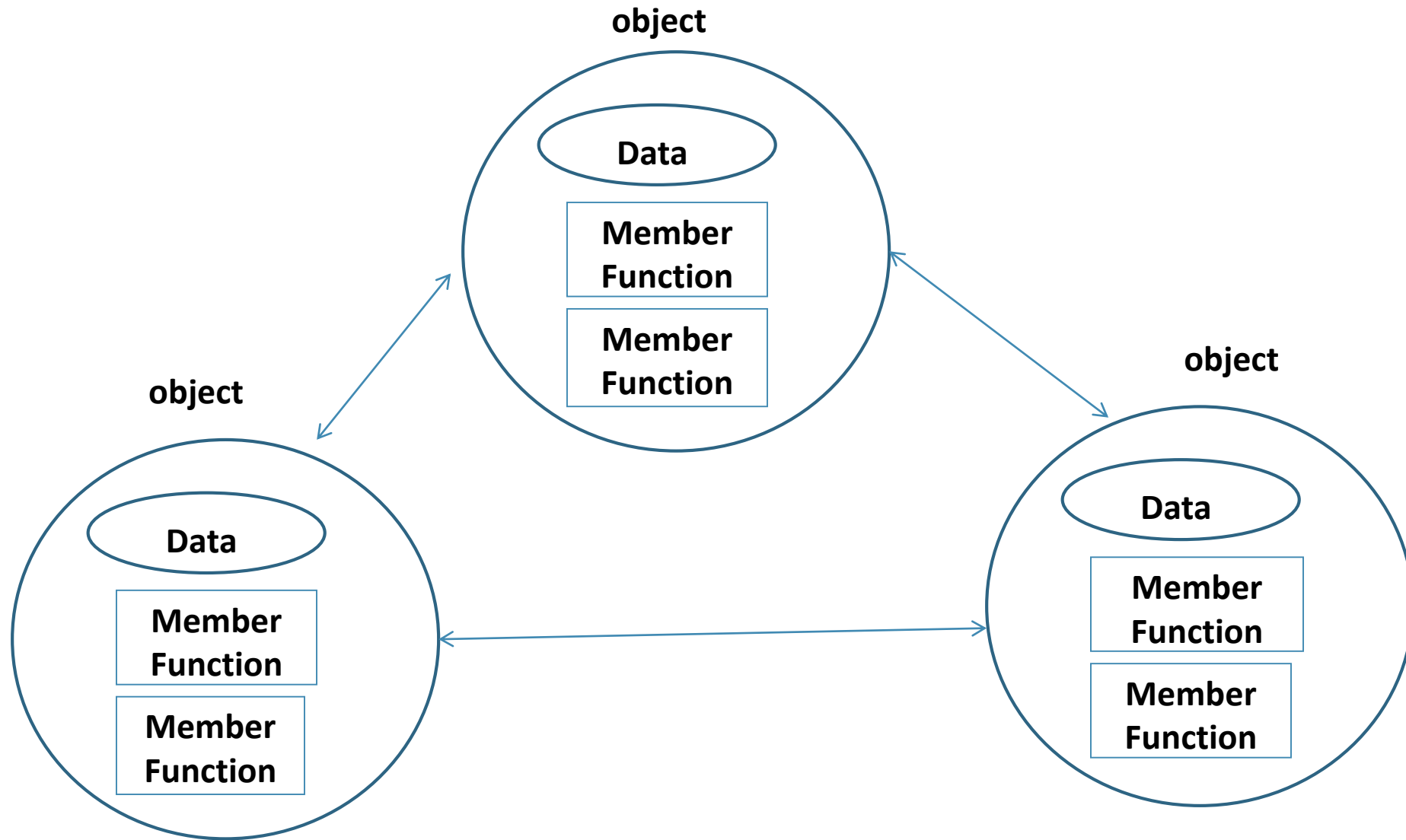
- An object's data is typically accessed through its member functions, i.e. it is hidden from accidental alteration
- Data and its function are said to be encapsulated into a single entity
- Data encapsulation and data hiding are key elements of object-oriented languages



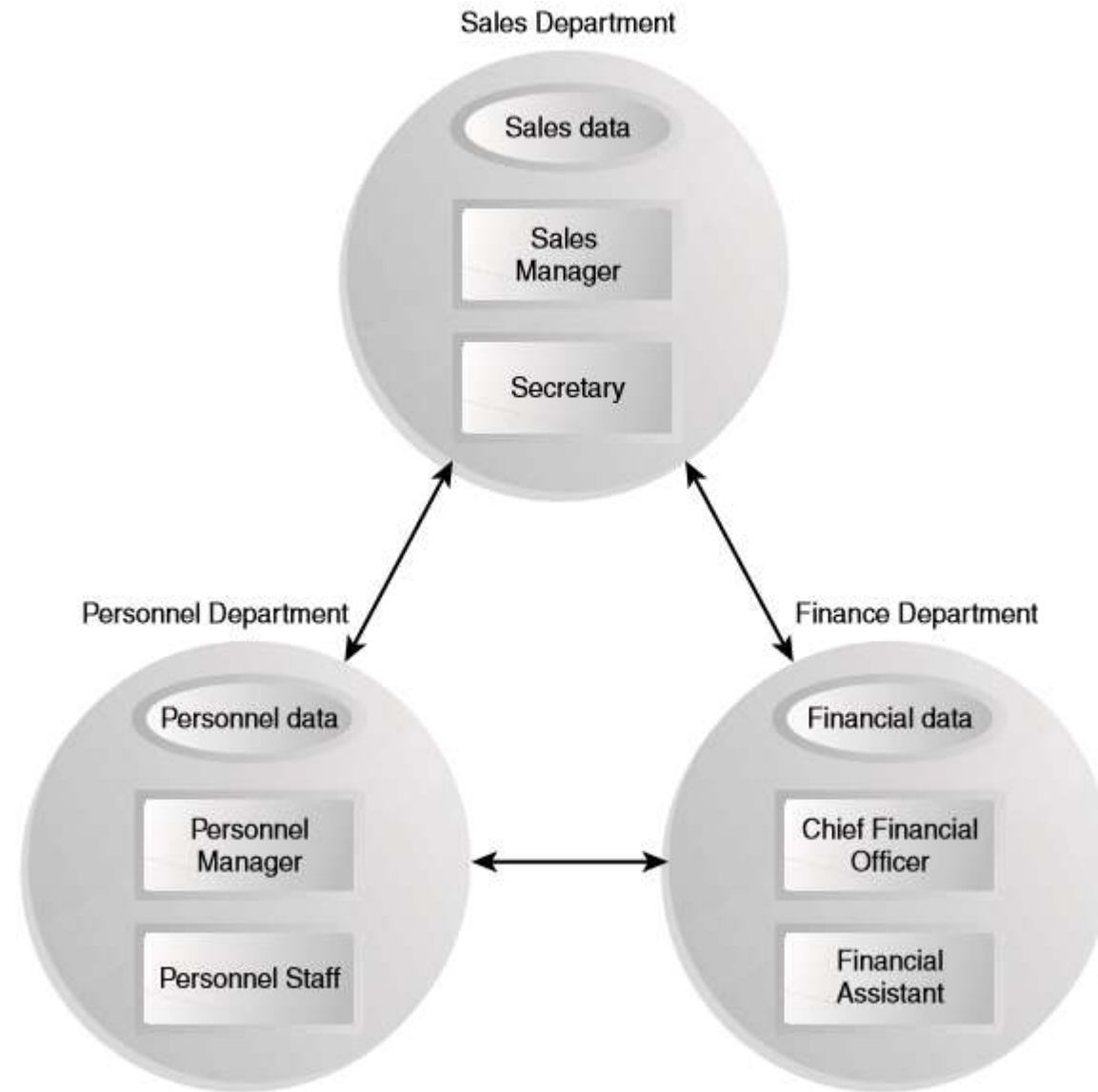
THE OO APPROACH

- If you want to modify data in an object, you know exactly what functions interact with it (i.e. the member functions of the object).
- This simplifies writing, debugging, and maintaining the programs
- An OO program consists of a number of objects which communicate with each other's member functions

THE OBJECT-ORIENTED PARADIGM

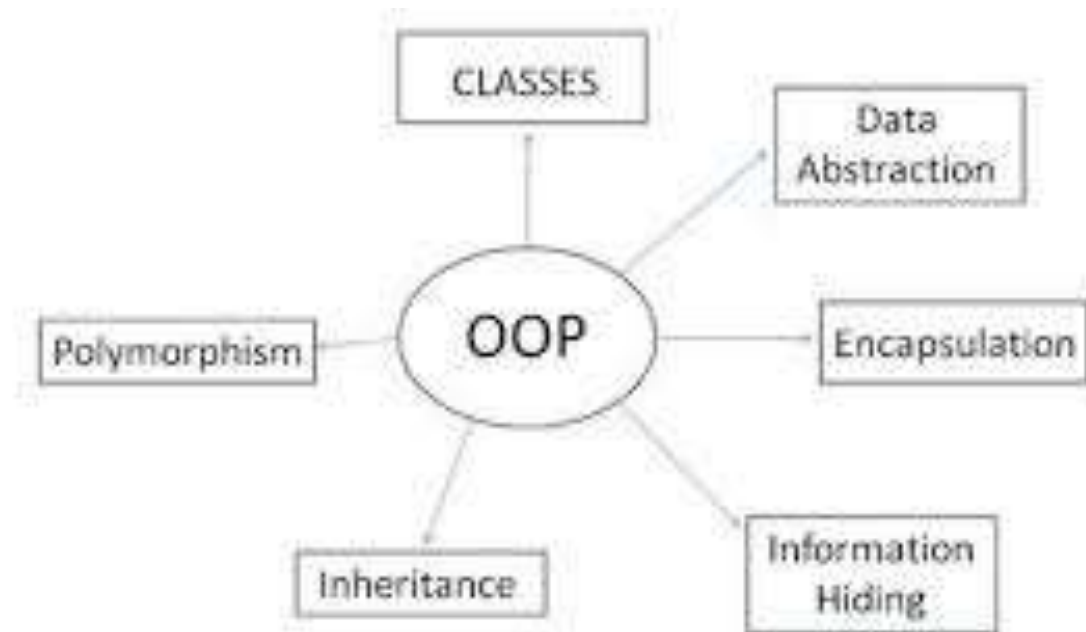


Example:



CHARACTERISTICS OF OO LANGUAGES

- **Objects**
- **Classes**
- Abstraction
- Encapsulation
- Inheritance
- Polymorphism and overloading



Objects Identification

- Physical objects
- Elements of the computer-user environment
- Data-storage constructs
- Human entities
- Collections of data
- User-defined data types
- Components in computer games

CLASSES

- Objects belong to classes
- Objects are members of classes.
- A class and an object of that class has the same relationship as a data type and a variable
- All objects with the same characteristics (data and functions) constitute one class.
- A class serves only as a plan, or a template, or sketch- of a number of similar things
- It merely specifies what data and what functions will be included in objects of that class.

CLASSES

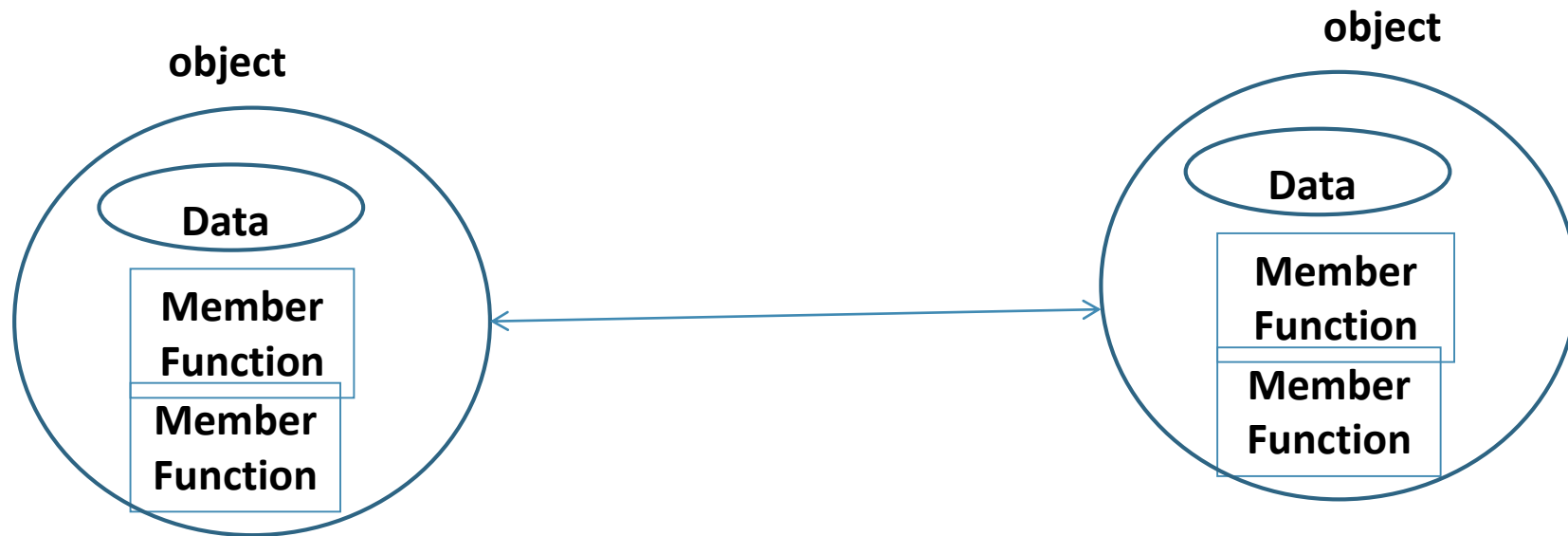
- Declaring a class doesn't create any objects, just as mere existence of data type `int` doesn't create any variables.
- A class is thus a description of a no. of similar objects.
- For instance, `HUMAN` is a class, and `JOHN` is its instance (object)

Abstraction

- Providing only essential information to the outside world and hiding their background details,
i.e., to represent the needed information in program without presenting the details.
- C++ classes provides different methods to the outside world without giving internal detail about those methods and data.

Encapsulation

- Information hiding
- Encapsulation is the mechanism that binds together code and the data it manipulates, and keep both safe from outside interference and missuse

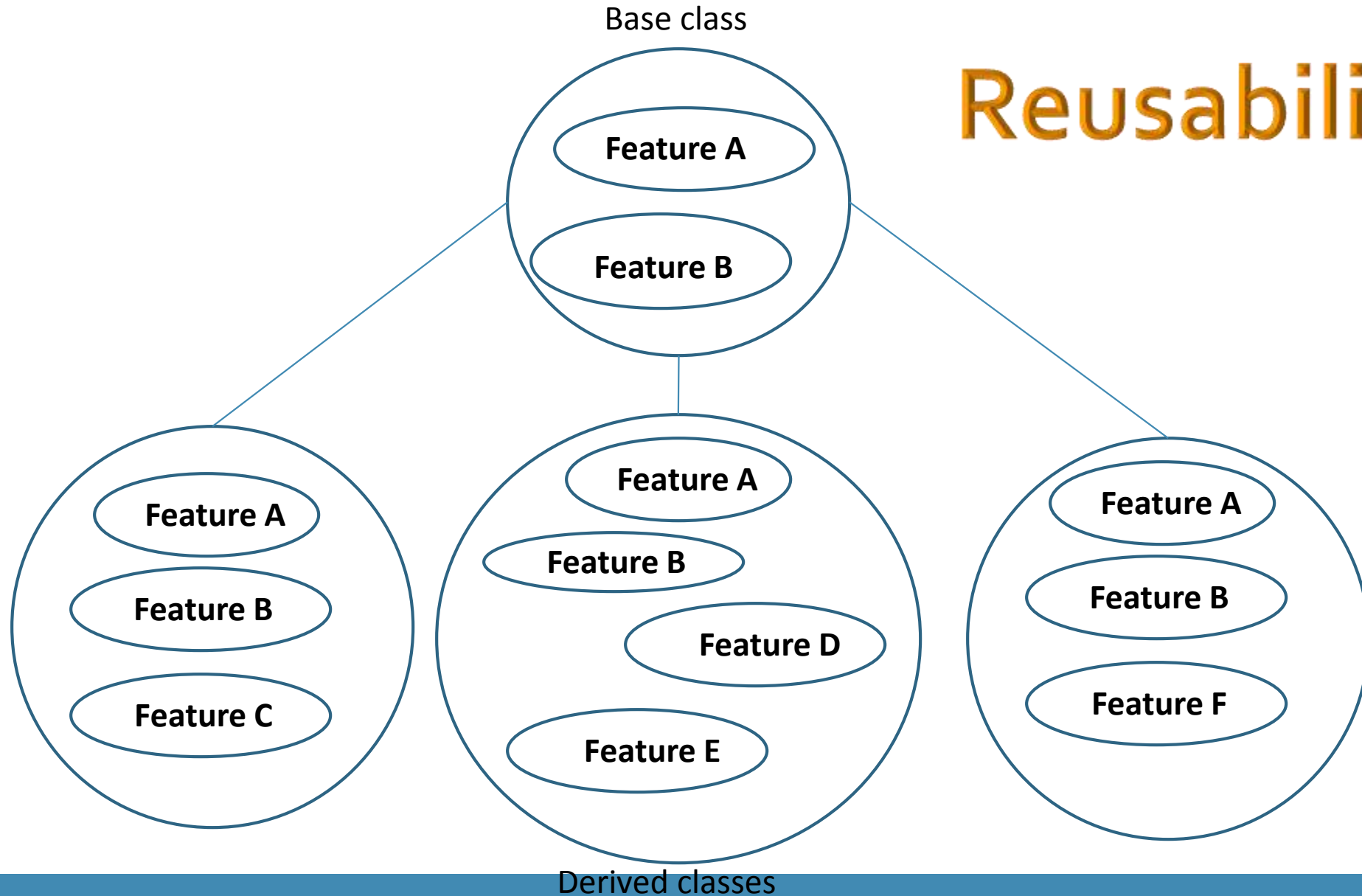


Inheritance

- Derive other (sub-)classes from an existing class
- The original class is called the BASE CLASS; the others are DERIVED CLASSES
- Each class shares common characteristics with the class from which it was derived, and can also add its own modifications, additions.
- For instance, VEHICLE is a class from which CAR, TRUCK, BUS, MOTORCYCLE classes can be derived.

INHERITANCE

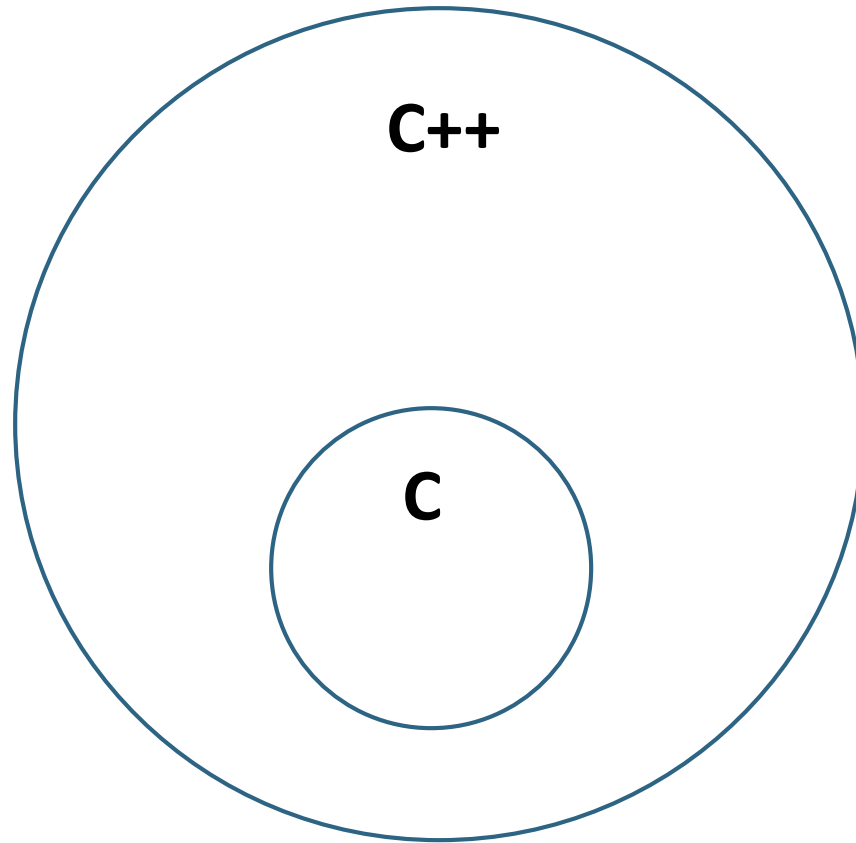
Reusability



POLYMORPHISM AND OVERLOADING

- Using operators or functions in different ways depending on what they are operating on is called polymorphism (lit. one thing with several distinct forms)
- Overloading is a special case of polymorphism, e.g. +, -, /, << etc.
- Ex: `int a, int b, int c → c=a+b;`
- `Obj a, obj b, obj c → c = a+ b; ????`

C AND C++



EXAMPLE PROGRAM IN C++

```
#include <iostream>

using namespace std;

int main()
{
    cout<<"Welcome to C++ Programming \n";
    return 0;
}
```

PROGRAM EXPLANATION

- `<iostream>`
 - This header supports C++ I/O operations.
 - `iostream` is to C++ what `stdio.h` is to C
- `using namespace std`
 - This tells the compiler to use the `std` name space
 - `std::cout`
 - This is the namespace in which the entire Standard C++ library is declared
 - This is required when we use the names that are brought into the program by the processor directive `#include<iostream>`
- `int main()`

PROGRAM EXPLANATION

- `cout`
 - An object
 - Predefined in C++ to correspond with the “standard output stream”
- Stream
 - An abstraction that refers to a flow of data
 - The standard output stream normally flows to the screen
- The Operator `<<`
 - Directs the contents of the variable on its right to the object on its left.

INPUT WITH cin

```
int main()
{
    int ftemp;
    cout<<"Enter temperature in fahrenheit: ";
    cin>>ftemp;
    int ctemp=(ftemp-32)*5/9;
    cout<<"Equivalent in celsius is : "<<ctemp;
    cout<<endl;
    return 0;
}
```

Quiz 1

1. Write one example for each. Procedural Language and OO Language.
2. The two major components of an object are .
3. In C++, a function contained within a class is called
 - a. a member function.
 - b. an operator.
 - c. a class function.
 - d. a method.
4. Bundling data and functions together is called _____.
5. Which of the following are good reasons to use an object-oriented language?
 - a. You can define your own data types.
 - b. Program statements are simpler than in procedural languages.
 - c. An OO program can be taught to correct its own errors.
 - d. It's easier to conceptualize an OO program.

1. The ability of a function or operator to act in different ways on different data types is call
2. A normal C++ operator that acts in special ways on newly defined data types is said to be
 - a. overriding.
 - b. encapsulated.
 - c. classified.
 - d. overloaded.
3. _____ serve as a blueprint to _____.

Thank You !